用户与鉴权的业务边界

gomall · 注册 / 双 token / RBAC / admin bootstrap / 三层鉴权路线图

---

gomall · 业务侧 deck

今日大纲

业务困局：鉴权不止是” 登录拿 token”

注册 + 登录 + 双 token：第一道业务边界

RBAC：30s 缓存为什么业务能接受

admin bootstrap + 三层边界路线图

业务场景：补券 / 业务码 / 客服话术

业务困局：鉴权不止是”登录拿token”

鉴权是商业模式的边界

- 一个电商，凡是和”钱”挂钩的接口，背后都是一句话：**这个请求是谁发的，他能做什么**。
- gomall 把这句话拆成三件事：
  - **身份**：注册 / 登录 / token 续期——客户在不在。
  - **角色**：user / merchant / admin ——客户能看见什么。
  - **边界**：路由分组 + middleware 链——客户做不到什么。
- 任何一件做错，损失都是真金白银：身份错则越权下单，角色错则客服把全场红包发给自己，边界错则 admin 接口暴露给 C 端。

routes/router.go:34–45 公开 vs authed；:148–155 admin 子组

gomall 现在的三层墙（实际只立起来两层）

- 公开层：能匿名访问，挂 HTTP cache，不挂任何 auth。
- authed 层：所有 C 端用户登录后能用，挂 `AuthMiddleware`。
- merchant 层：**当前还没有独立 group**，商家身份借 user 表 Role 字段表达。
- admin 层：在 authed 内嵌一层 `RequireRole("admin")`。

routes/router.go:39–45 公开；:47–48 authed group；:148–150 admin sub-group

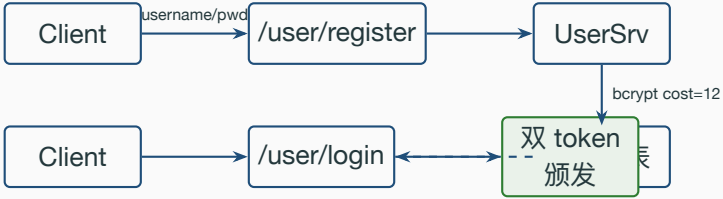
业务问题清单 + 本 deck 的回答顺序

1. 注册 + 登录链路，密码加密 / 邮箱验证业务边界画在哪。
2. access 24h / refresh 10d 这两个数字怎么定，怎么影响留存。
3. RBAC 为什么敢上 30s 内存缓存。
4. admin bootstrap 这个” 一次性接口” 为什么必须存在。
5. C 端 / merchant / admin 三层边界，merchant 还差什么。
6. 客服补券走哪条路？为什么用户自助补券是黑产温床。
7. 业务码 `30001` / `30002` / `30005` 对应的客服话术。

鉴权是**业务对” 信任” 的一次明确表态**：信谁，信多久，信他能干什么。

注册 + 登录 + 双 token：第一道业务边界

注册 / 登录 / 颁发 token 的时序



routes/router.go:34–35 路由；

service/user.go:38–82 注册；*:85–121* 登录；*repository/db/model/user.go:28 PassWordCost=12*

密码是怎么存的：bcrypt cost=12

- **bcrypt** 自带 salt，单向，不可还原。重置密码只能让用户重新设置。
- `PassWordCost = 12`：单次哈希 ~250ms（M 系列 Mac）。
  - 业务收益：撞库脚本一秒只能猜 4 次，10 位密码组合在合理时间内不可穷举。
  - 业务代价：登录接口 RT 多 250ms，C 端可接受。
- 注册存哈希，登录拿明文重算 hash 再 compare。**后台永远拿不到明文**，被拖库后撞库代价巨大。
- 找回密码必须走**邮箱验证 + token 写新摘要**：`EmailClaims` 携带 bcrypt 摘要而非明文，TTL **15min** 远短于 access。

repository/db/model/user.go:34–48 SetPassword / CheckPassword；service/user.go:56 注册调用；pkg/utils/jwt/jwt.go:95–134 EmailClaims

双 token 续期的状态机

- **access valid**：`AuthMiddleware` 顺便**续发**新 access + refresh，写回 header + cookie。
- **access 过期 / refresh 有效**：拿 refresh 重发一对新 token，用户无感知。
- **两者都过期**：返回 `30001`，C 端跳登录页。

pkg/utils/jwt/jwt.go:67–93 ParseRefreshToken；middleware/jwt.go:59 SetToken；consts/user.go:31–32 24h/10d

24h / 10d 这两个数字的业务依据

参数	取值	业务依据
access 短时窗	24h	单日活跃无需重登；token 泄露最长 24h
refresh 长时窗	10d	周末 + 黄金周回流用户仍能静默续期
邮箱 token	15min	邮件链路被截短 TTL 反制
RBAC 缓存	30s	提权 / 降权延迟可接受

- 10d 是**留存指标的妥协**：app 端” 打开就在线” → 用户不流失。
- access 24h 不能更长：移动端被刷机 / 偷设备时，攻击窗口必须有限。
- 不做 7d：周中休假 + 周末” 双周回归” 用户会被强制重登，体验差。

consts/user.go:31–32 AccessTokenExpireDuration / RefreshTokenExpireDuration

代码片段 1：AuthMiddleware 主链路

Listing 1: *middleware/jwt.go:16–64 AuthMiddleware*

```
accessToken := c.GetHeader("access_token")
refreshToken := c.GetHeader("refresh_token")
if accessToken == "" {
   code = e.InvalidParams
   c.JSON(200, gin.H{"status": code, ...})
   c.Abort(); return
}
newAccessToken, newRefreshToken, err :=
   util.ParseRefreshToken(accessToken, refreshToken)
if err != nil { code = e.ErrorAuthCheckTokenFail }
...
SetToken(c, newAccessToken, newRefreshToken)
c.Request = c.Request.WithContext(
   ctl.NewContext(c.Request.Context(), &ctl.UserInfo{Id: claims.ID}))
```

逐行讲解 1：AuthMiddleware

- `GetHeader` 两个 token ——C 端 SDK 约定都从 header 拿，不解析 cookie（避免与第三方网关 cookie 同名冲突）。
- `accessToken == ""` 直接 `400 InvalidParams`，**不是 30001**。区分” 完全没带” vs ” 带了但错”，便于客服判断是不是 SDK bug。
- `ParseRefreshToken` 一次出双 token：每次请求都续期，access 永远 24h 起跳。
- `SetToken` 同时写 `header + cookie + SameSiteLax`：兼容 SPA 和老 web 多窗口。
- 潜在坑：续期 token 写 header 后，C 端必须**用最新 header 覆盖本地存储**，否则下一次还是老 token。
- 可改进：当前 `ParseRefreshToken` 把”access 没过期” 也当作续期信号，浪费一次 HS256 签名。可加” 剩余 < 1h 才续” 的判断。

middleware/jwt.go:20–62

双 token 不查黑名单 + 鉴权链路开销

场景	RPS	p95
/ping （基线）	64,254	3.51ms
/orders/list （带 Auth）	58,406	5.00ms

鉴权链路开销 $\approx 1 - 58406/64254 = 9.1\%$，

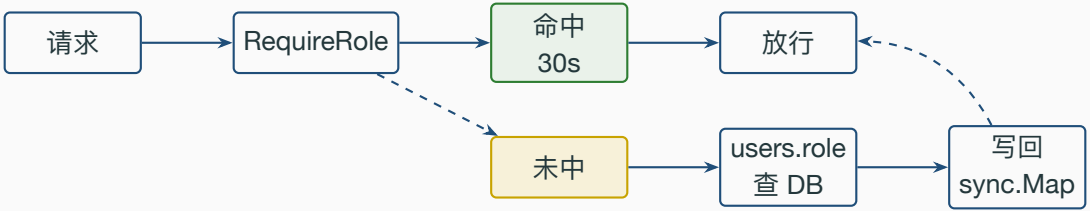
敢挂 authed 全组。HS256 纯 CPU 无 IO。

- 当前**不维护 token 黑名单**：登出 = 客户端丢 token，服务端不主动作废。
- 后果：被偷设备最多多用 24h；改密码后老 access 仍能用 24h。
- 敢这么做的前提：token 仅控访问，**不直接控钱**；提现 / 大额支付有二次验证。
- 路线图：登出 / 改密码事件写 Redis 黑名单 set（TTL = access 剩余），鉴权链路多一次 `SISMEMBER`。

stressTest/REPORT.md 链路压测表行 *1, 4*；*middleware/jwt.go:32–57* 当前无黑名单

RBAC：30s 缓存为什么业务能接受

RBAC 30s 内存缓存命中链路



middleware/rbac.go:24 roleCacheTTL=30s；:28–45 lookupRole；:48–86 RequireRole

代码片段 2：lookupRole + 30s sync.Map 缓存

Listing 2: *middleware/rbac.go:22–45*

```go
var (
   roleCache sync.Map
   roleCacheTTL = 30 * time.Second
)
func lookupRole(ctx context.Context, userId uint) (string, error) {
   if v, ok := roleCache.Load(userId); ok {
      entry := v.(roleCacheEntry)
      if time.Now().Before(entry.expires) {
         return entry.role, nil
      }
   }
   u, err := dao.NewUserDao(ctx).GetUserById(userId)
   if err != nil { return "", err }
   role := u.Role
   if role == "" { role = "user" }
   roleCache.Store(userId,
      roleCacheEntry{role, time.Now().Add(roleCacheTTL)})
   return role, nil
}
```

逐行讲解 2：lookupRole

- `sync.Map`：**读多写少**的典型场景，比 `RWMutex+map` 在高并发读下吞吐更高。
- `Load` 走 ConcurrentMap fast-path；命中后**零锁**返回。
- `role == ""` 的兜底是**业务安全网**：早期老用户 Role 字段是空字符串，强制视为最低权限 user。
- 写回不加锁：sync.Map.Store 内部 atomic，并发覆盖也只是同值，业务无影响。
- 潜在坑：进程重启缓存清零，第一波 admin 请求会全部回源 DB。单实例影响很小。
- **为什么不用 Redis**：本机 sync.Map 是**零 RTT**，鉴权延迟低于 1ms；Redis 反而引入网络依赖。

30s 在业务上意味着什么 + 显式失效兜底

- 提权场景：管理员把客服 A 升为 admin → 最长 30s 后享受 admin 权限。
- 降权场景：admin 误授权后回收 → 误授权用户最长 30s 内仍能继续操作。
- 30s 误授权窗口能干坏事吗?
  - admin 当前接口（users / promote / backfill）不会瞬间造成资金损失。
  - 真正控钱的接口（提现 / 退款）尚未挂在 admin 子组，无 30s 资金风险。
- 必须零 TTL 的场景（封禁恶意 admin），用 `InvalidateRoleCache(userId)` **显式失效**。
- 选型 = **最终一致 + 关键路径显式失效**：99% 走 0ms 缓存命中，1% 走 Invalidate 立即生效。

middleware/rbac.go:88–91 InvalidateRoleCache；service/admin.go:47–62 PromoteToAdmin 写完即 Invalidate

admin bootstrap + 三层边界路线图

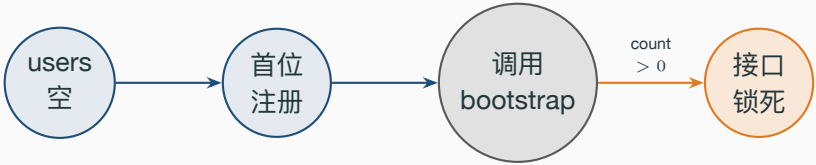
冷启动悖论

- 上线一个全新 gomall 实例，users 表是空的。
- 想给”运维大佬”开 admin 权限，必须调 `/admin/users/promote`。
- 但 `/admin/...` 子组的中间件链是 `Auth + RequireRole("admin")`。
- **此时一个 admin 都没有，谁也进不去 admin 子组** → 没人能把别人提升为 admin。

> 这就是 admin bootstrap 必须存在的原因：**先有第一个 admin，后面 RBAC 才能转起来。**

routes/router.go:146 admin/bootstrap 仅挂 authed，无 RequireRole；service/admin.go:66–80 BootstrapPromoteSelf

冷启动状态机



- **核心校验**：`BootstrapPromoteSelf` 在执行前 `SELECT count(*) WHERE role='admin'`。
- count $= 0$：把当前调用者升为 admin。
- count > 0：返回错误” 系统已存在 admin”，**接口从此永久不可用**。
- 路由层做法：`authed.POST("admin/bootstrap")`，仅在 `AuthMiddleware` 之下，但**不**挂 `RequireRole`。

service/admin.go:72–78 count 校验；routes/router.go:146

代码片段 3：BootstrapPromoteSelf 一次性校验

Listing 3: *service/admin.go:66–80 BootstrapPromoteSelf*

```go
func (s *AdminSrv) BootstrapPromoteSelf(ctx context.Context) error {
    u, err := ctl.GetUserInfo(ctx)
    if err != nil { return err }
    db := dao.NewUserDao(ctx).DB
    var count int64
    if err := db.Model(&model.User{}).
        Where("role = ?", model.RoleAdmin).
        Count(&count).Error; err != nil {
        return err
    }
    if count > 0 {
        return errors.New("系统已存在 admin，禁止使用 bootstrap 接口")
    }
    return s.PromoteToAdmin(ctx, u.Id)
}
```

逐行讲解 3：BootstrapPromoteSelf

- `ctl.GetUserInfo` 取的是 `AuthMiddleware` 写进 ctx 的 user id ——**调用者必须已登录**。匿名打 bootstrap 会先被 AuthMiddleware 截掉。
- `count(*) WHERE role='admin'`：业务**自检**。`users.role` 有 index，实际是 range scan，毫秒级。
- `count > 0 return err`：这一行就是**接口"自锁"机制**。
- `PromoteToAdmin`：复用提权逻辑，触发 `InvalidateRoleCache` 立即生效。
- 潜在并发坑：两人同时打 bootstrap，count=0 都通过 → 两人都被提为 admin。**第一次部署窗口**通常只有运维一人，可接受。真要修：`SELECT ... FOR UPDATE` 或表级 advisory lock。

C 端 / merchant / admin 三层对比

角色	路由位置	中间件链	业务能力
匿名 user	v1 顶层	仅全局桶	浏览商品
C 端 user	authed 组	+ AuthMiddleware	下单 / 抢券
merchant	暂无	—	**路线图**
admin	admin 子组	+ RequireRole	提权 / 后台

- C 端 vs admin 边界靠 `RequireRole("admin")` 拦下。
- merchant 还没独立 group ——商家身份只是 user 表 Role 字段的占位。
- product/create 现在在 authed 组：任意已登录用户都能调，**没有商家校验**。

routes/router.go:47–48 authed；:61–63 product/create 待迁；:148–155 admin

merchant group 的路线图：路由 + DAO 双层

1. authed 内增设 `merchant := authed.Group("/merchant")`，挂 `RequireRole("merchant","admin")`。
2. 商品 CRUD / 库存 / 订单导出全部迁过来。
3. users 表加 `shop_id` 外键，DAO 层每个 query 强制带 `shop_id = current_user.shop_id`。

> RBAC 解决”**他能不能进这个路由**”；shop_id 解决”**他能不能动这条记录**”。两者缺一不可。

repository/db/model/user.go:23 Role 字段已就绪；routes/router.go:61–63 待迁移

业务场景：补券 / 业务码 / 客服话术

客服补券的路径选择

方案	路径	黑产风险	审计
A. 用户自助补	C 端调 /coupon/claim	**高**	无
B. 客服走 admin	admin 后台接口	低	有

- **A 不行**：用户自助意味着 token 持有者就能领，黑产**万号脚本** 1 分钟领光所有券。
- **B 是标准**：补券必须由**有 admin 权限的客服**发起，每次请求都**留 operator_id 进审计日志**。
- gomall 当前 admin 子组只有 promote / backfill，**补券接口待补**。
- 路线图：`admin.POST("coupon/grant", ...)` 接收 target_user_id + reason，写 admin_audit 表。

routes/router.go:148–155 admin 子组当前只有 3 个接口；deck 10 SlidingWindow 防自助补券黑产

业务码 + 客服话术（鉴权三种姿势）

码	含义	客服话术
400 InvalidParams	token 没带 / 格式错	” 请更新最新版 app 再试”
30001 ErrorAuthCheckTokenFail	token 解析失败	” 请重新登录”
30002 ErrorAuthCheckTokenTimeout	双 token 都过期	” 您已超过 10 天未登录，请
30005 ErrorAuthInsufficientAuthority	角色不够	” 该操作需要管理员权限”

- 400 vs 30001：客服区分”**SDK bug** vs **token 真坏**”。
- 30001 vs 30002：用户视角都是” 要重登”，但客服能区分” 账户安全事件 vs 正常长闲置”。如果某用户 30001/30002 比例异常，可能是被换设备登录，触发安全审计。
- 30005：admin 接口必备——普通用户摸 admin 时给清晰提示。

pkg/e/code.go:35–58；middleware/jwt.go:24,38,51 三处 c.JSON(200,...)；middleware/rbac.go:77

鉴权路线图（按优先级）

1. **P0**：登出 / 改密码 token 黑名单（Redis set + TTL）。
2. **P0**：admin 操作审计表 admin_audit（operator / target / reason / ts）。
3. **P1**：merchant 独立 group + DAO 层 shop_id 强制隔离。
4. **P1**：bootstrap 接口加 advisory lock，防并发提权。
5. **P2**：admin 子组的” 补券 / 退款 / 封号” 标准接口。
6. **P2**：access 续期改” 剩余 < 1h 才续”，省一次 HS256。

鉴权不是” 加一个 if”，是**用最少的代码代价表达最严的业务边界。**

middleware/jwt.go：待加黑名单；*service/admin.go*：待加 *advisory lock*；*routes/router.go*：待加 *merchant group*

面试 Q&A（上）

Q1：access 24h / refresh 10d 这两个数字怎么定？

A：access 短到能限制 token 泄露损失，refresh 长到覆盖周末回流用户。10d 是留存指标和安全的折中。

Q2：RBAC 30s 内存缓存，admin 误授权 30s 不是漏洞吗？

A：30s 是常规延迟。关键路径调 `InvalidateRoleCache(userId)` 显式失效，提权 / 降权立即生效。

Q3：双 token 不查黑名单，登出后老 token 还能用 24h 怎么办？

A：当前 token 仅控访问不直接控钱，大额操作有二次验证。路线图里加 Redis 黑名单。

面试 Q&A（下）

Q4：admin bootstrap 一次性接口怎么防止重复使用？

A：`SELECT count(*) WHERE role='admin'`，count > 0 直接返回错误。接口本身存在，但永远进不到提权那一步。

Q5：merchant 当前没独立 group，为什么不阻塞上线？

A：单店家 / 自营场景下 user 即 merchant，业务上不区分。多店家路线图阶段才需要 group + shop_id 双层隔离。

Q6：业务码 30001 vs 30002 用户视角一样，区分有什么意义？

A：用户视角一样（都要重登），但客服后台可以区分” 账户安全事件 vs 正常长闲置”。比例异常时触发安全审计。

代码位置一览

authed group 入口 *routes/router.go:47–48*

admin sub-group *routes/router.go:148–155*

admin/bootstrap 路由 *routes/router.go:146*

AuthMiddleware *middleware/jwt.go:16–64*

SetToken 写回 *middleware/jwt.go:66–73*

双 token 续期 *pkg/utils/jwt/jwt.go:67–93*

邮箱 token 15min *pkg/utils/jwt/jwt.go:95–134*

RequireRole + 30s 缓存 *middleware/rbac.go:22–45, 48–86*

InvalidateRoleCache *middleware/rbac.go:88–91*

BootstrapPromoteSelf *service/admin.go:66–80*

PromoteToAdmin *service/admin.go:47–62*

Role 字段 / bcrypt cost=12 *repository/db/model/user.go:23, 28, 30–31, 34–48*

业务码定义 *pkg/e/code.go:35–58*

access/refresh 常量 *consts/user.go:31–32*